Conditional-TimeGAN for Realistic and High-Quality Appliance Trajectories Generation and Data Augmentation in Nonintrusive Load Monitoring

Z. G. Liu, T. Y. Ji, *Senior Member, IEEE*, J. W. Chen, L. J. Zhang, L. L. Zhang, *Member, IEEE*, and Q. H. Wu, *Life Fellow, IEEE*

***Abstract*— Nonintrusive load monitoring (NILM) strives to achieve real-time monitoring of individual appliance energy consumption and usage by leveraging aggregate power readings. Most of the existing NILM models are based on machine learning. To improve the generalization and accuracy of NILM models, it is crucial to expand the dataset. However, collecting a large amount of power data is a challenging and common task. To address this, we propose a novel model called conditional-time-series generative adversarial network (C-TimeGAN), which extends upon TimeGAN by incorporating constraint conditions to generate high-quality and realistic appliance trajectories. This model preserves the benefits of unsupervised GANs' flexibility and supervised training's stepwise control in TimeGAN while addressing challenges in appliance data generation, such as rapid power changes and transients. In addition, we utilize a one-class support vector machine (OCSVM) for postprocessing to further enhance data quality by detecting anomalies and removing outliers. Experiments show that the synthetic data from our C-TimeGAN$^+$ model (C-TimeGAN integrated with OCSVM) outperform the baseline in terms of both quality and quantity, exhibiting higher fidelity, diversity, and novelty. Furthermore, C-TimeGAN$^+$ serves as a practical data augmentation tool, enhancing disaggregation accuracy and generalization of other NILM models. We also discuss the optimal proportion of synthetic data for augmentation, improving the practical applicability of the data.**

***Index Terms*— Conditional-time-series generative adversarial network (C-TimeGAN), data augmentation, data generation, energy disaggregation, nonintrusive load monitoring (NILM), one-class support vector machine (OCSVM).**

I. Introduction

EFFECTIVE electricity load monitoring is a critical component of sustainable energy management. By precisely tracking the energy usage of individual appliances in homes and buildings, energy providers and consumers can identify areas of high consumption and implement tailored conservation strategies. This can result in significant cost savings and reduction of greenhouse gas emissions while promoting greater awareness of environmental responsibility [1].

Nonintrusive load monitoring (NILM) facilitates real-time monitoring of energy consumption and usage for individual appliances within a power-consuming unit. This is accomplished through the installation of a smart sensor module at the entry point of the unit's electricity supply [2]. Since the inception and development of NILM systems in 1992, various algorithms have been proposed, including fuzzy algorithm [3], hidden Markov models [4], adaptive boosting (Adaboost) [5], and neural networks [6], [7], among others. However, the precise estimation of the appliance power consumption distribution remains the primary challenge for most supervised NILM algorithms. Specifically, the posterior distribution of each appliance depends on the total power measurement [8]. Therefore, the efficacy of supervised methods significantly hinges on the degree of alignment between the distribution of training and actual data.

As demand for load monitoring accuracy has grown, the complexity of NILM models has experienced a substantial uptick [9]. Hence, there is a need to proportionally expand the training dataset. Nonetheless, researchers often encounter a shared obstacle stemming from the scarcity of adequately extensive data samples for training. Thus, the expansion of appliance data sample size has emerged as a pivotal focal point in advancing research within the realm of NILM.

Expanding the sample dataset is crucial to increase the accuracy and generalization of the NILM model. The central aim centers around enriching the electricity consumption data for each distinct appliance, a goal achieved through two prevalent approaches: data collection [10], [11] and data generation, involving the synthesis of artificial data [8].

The process of data collection not only demands considerable effort from researchers but also encounters substantial challenges in accurately capturing data from scenarios involving the simultaneous usage of multiple appliances. This difficulty is exacerbated by the variability in factors such as appliance usage duration, sampling frequency, and diverse instrumentations for data collection. Moreover, the

Manuscript received 24 August 2023; revised 21 February 2024; accepted 8 March 2024. Date of publication 26 March 2024; date of current version 5 April 2024. This work was supported in part by the Guangzhou Science and Technology Planning Project under Grant 202102020688 and in part by the Natural Science Foundation of Guangdong Province under Grant 2022A1515011608. The Associate Editor coordinating the review process was Dr. Benkuan Wang. *(Corresponding author: L.L. Zhang.)*

The authors are with the School of Electric Power Engineering, South China University of Technology, Guangzhou 510641, China (e-mail: epzhangll@scut.edu.cn).

Digital Object Identifier 10.1109/TIM.2024.3381263

time-intensive nature of gathering real-world data, coupled with financial and privacy concerns, poses major obstacles in acquiring a sufficiently large and diverse dataset.

In this context, the necessity of synthetic data becomes glaringly evident. Synthetic data, generated through advanced algorithms such as generative adversarial network (GAN) models, can mimic the characteristics of real-world appliance usage data, providing a rich and diverse training ground for NILM models. By augmenting the limited real data with high-quality synthetic data, researchers can effectively expand the sample size of appliance data. This augmentation not only compensates for the scarcity of real-world datasets but also introduces a wider range of load scenarios and variations, which are crucial for training more robust and accurate NILM models.

In our previous work [12], we fully considered the advantages of time-series GAN (TimeGAN) in generating temporal data and successfully utilized it to synthesize both realistic and high-quality NILM data. In this article, we extend TimeGAN with additional constraints to form conditional-TimeGAN (C-TimeGAN) and apply the one-class support vector machine (OCSVM) postprocessing method to specifically address the unique challenges in NILM data generation, including appliance interdependency, subtle and rapid power fluctuations, data realism and variability, and measurement bias immunity, thereby further improving the quality of the generated data. Furthermore, our improved C-TimeGAN$^+$ model (C-TimeGAN integrated with OCSVM) functions as a practical data augmentation tool, enhancing disaggregation accuracy and the generalization of other NILM models. We also delve into the optimal proportion of synthetic data for augmentation, improving the practical applicability of this approach. Our contributions are as follows.

1) C-TimeGAN is a novel extension of TimeGAN that introduces appliance-related coupling constraints and first-order power difference constraints tailored to capture the subtle and rapid power variations in data dynamics, making the model more suitable for NILM high-quality and realistic data generation. This advanced generation stands out not only for its cost-effectiveness compared to traditional instrumentation and measurement methods but also for its immunity to measurement-related biases [13], [14].
2) To further refine the quality of synthetic data generated by C-TimeGAN, the addition of OCSVM aids in detecting anomalous sequences and filtering out outliers. This data postprocessing and refinement not only ensure that the data generated by C-TimeGAN surpass similar GAN-based models in both qualitative and quantitative aspects but also provide insights into addressing the challenge of enhancing data accuracy and reliability in instrumentation and measurement [15], [16].
3) Utilizing the synthetic data produced by C-TimeGAN as a tool for data augmentation enhances the precision and generalizability of other NILM models. This augmentation results in improved outcomes in energy disaggregation tasks, contributing to more accurate and reliable predictions. These advancements directly address instrumentation and measurement challenges such as data scarcity, model overfitting, and limited model applicability for other NILM models [6], [17], [18]. C-TimeGAN's high-fidelity synthetic datasets support robust NILM model development, ensuring adaptability and sustained high performance across diverse scenarios.

II. Related Work

A. Generative Adversarial Networks (GANs)

Deep neural networks have shown great success in the NILM field for tasks such as classification and regression [6], [7]. However, generating synthetic data has been a challenge for these models until the introduction of GAN. In 2014, Goodfellow et al. [19] proposed GAN as a new generative model that outperforms the variational self-encoder-based generative models in terms of generative effect.

In recent years, GAN modeling methods have found applications in the smart grid domain, particularly in the field of NILM [20]. Researchers have explored the potential of conditional GANs (CGANs) in NILM, with Pan et al. [21] using power data from a target appliance as real samples and the bus power data as conditions to guide the generative and discriminative models. This approach enables the disaggregation of bus power data into individual appliance power data through adversarial training. Ahmed et al. [22] proposed two GAN models for NILM: one is GAN-NILM using parameter-sharing transfer learning across households and the other is transfer-GAN (TrGAN)-NILM to minimize statistical distance in the feature space.

B. Data Generation

To address the issue of limited sample size in NILM, there are two main methods. The first involves expanding the original samples by overlaying and interpolating them with new samples. For instance, Huang et al. [23] applied Gaussian noise to the original dataset to generate new samples for neural network training. However, the uncertainty of the generated data makes it unsuitable for deep learning tasks in the NILM domain. Chawla et al. [24] proposed the synthetic minority oversampling technique (SMOTE), which addresses the limited sample size issue by generating synthetic data through interpolation. However, this method often results in overlapping class samples, leading to weakened classification capabilities of the model.

Another type of method is based on statistical models, machine learning, and deep learning models. He et al. [25] introduced a support vector machine (SVM)-based method for sample data generation, where the existing samples are used to train an SVM model with optimal parameters determined through grid search. However, selecting appropriate SVM parameters and kernel functions remains a significant challenge. Wang et al. [26] proposed a data generation method using variational autoencoders (VAEs), which has the advantage of fast convergence during training and no probabilistic modeling. Their synthesized data are diverse and can fully reflect the actual data itself.

Meanwhile, more and more researchers have been employing GAN for data synthesis. Radford et al. [27] introduced a novel approach, known as a deep convolutional GAN (DCGAN), which integrates the power of GANs with deep convolutional neural networks. This structure effectively addresses the training instability problem in GAN models and produces high-quality synthetic data. Harell et al. [8] introduced TraceGAN, a novel model generating authentic random data for NILM. This model uses incremental training and a conditioning mechanism, employing a 1-D Wasserstein generator to create signatures for various appliances. Unlike traditional GANs relying solely on adversarial feedback, they may struggle to capture power data's dynamic characteristics, potentially hindering high-quality appliance power data synthesis for NILM.

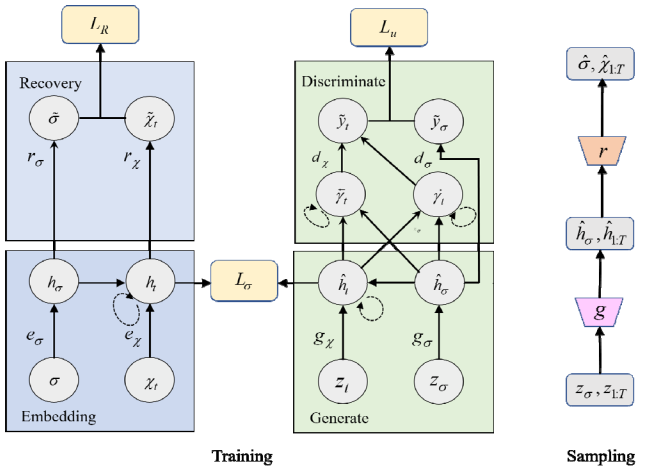

Fig. 1. Framework of TimeGAN.

III. Proposed Method

A. TimeGAN

TimeGAN [28] is an effective solution for NILM data due to its ability to incorporate both static and dynamic features for each temporal data point during training. This approach is particularly advantageous because the steady-state processes of appliances such as fridges can be considered static features, whereas the transients or fast switching ON/OFF states of appliances such as dishwashers can be viewed as dynamic features.

In fact, TimeGAN comprises two objectives, a global and a local one

$$\min_{\hat{p}} D\left(\left(p\left(\sigma, \chi_{1:T}\right)\right) \parallel \hat{p}\left(p\left(\sigma, \chi_{1:T}\right)\right)\right) \tag{1}$$

$$\min_{\hat{p}} D\left(\left(p\left(\chi_t \mid \sigma, \chi_{1:t-1}\right)\right) \parallel \hat{p}\left(\chi_t \mid \sigma, \chi_{1:t-1}\right)\right) \tag{2}$$

where the operator D represents a distance metric used to quantify the divergence between two probability distributions. Within the scope of GAN framework, this measure embodies the Jensen–Shannon divergence for the global objective in (1). For the local objective in (2), when leveraging the actual dataset for supervision through a maximum likelihood approach, it assumes the form of Kullback–Leibler divergence. σ and $\chi_{1:T}$ denote the vector space of static and dynamic features of time series data.

The global objective aims to learn a density $\hat{p}(\sigma, \chi_{1:T})$ that approximates the overall distribution $p(\sigma, \chi_{1:T})$ of the data, as shown in (1). Meanwhile, the local objective is to train a density $\hat{p}(\chi_t|\sigma, \chi_{1:t-1})$ that matches the step-wise distribution $p(\chi_t|\sigma, \chi_{1:t-1})$ at each time t, as indicated in (2). To achieve this, an autoregressive decomposition mechanism is introduced to handle the joint $p(\sigma, \chi_{1:T}) = p(\sigma)\prod_t p(\chi_t|\sigma, \chi_{1:t-1})$.

The framework of TimeGAN, as shown in Fig. 1, can be summarized as follows.

1) Embedding network mapping the input features to corresponding latent representations

$$h_\sigma = e_\sigma(\sigma), \quad h_t = e_\chi\left(h_\sigma, h_{t-1}, \chi_t\right) \tag{3}$$

where h_σ and h_t denote the corresponding latent spaces of σ and χ_T, respectively, with lower dimensionality and higher density of features. The function e represents the embedding of σ and χ_T features, trained using a recurrent neural network (RNN).

2) Recovery network reconstructing the original features from the compressed latent codes

$$\tilde{\sigma} = r_\sigma\left(h_\sigma\right), \quad \tilde{\chi} = r_x(h_t) \tag{4}$$

where $\tilde{\sigma}$ and $\tilde{\chi}$ represent the precise reconstructed features of σ and χ_T. The function r represents the recovery network for embedding latent spaces of h_σ and h_t, trained by an autoregressive network that follows causal ordering.

3) *Sequence Generator and Discriminator:* To enhance efficiency and accuracy, the generator's synthetic outputs are fed directly into the embedding space rather than the feature space. Suppose two vector spaces, $\mathbb{Z}_\sigma$ for static and $\mathbb{Z}_\chi$ for temporal, both with known distributions. Random vectors are sampled from these spaces to serve as input for generating the spaces $\mathbb{H}_\sigma$ and $\mathbb{H}_\chi$. The generator's function $g : \mathbb{Z}_\sigma \times \prod_{t=1}^{T} \mathbb{Z}_\chi \to \mathbb{H}_\sigma \times \prod_{t=1}^{T} \mathbb{H}_\chi$ accepts a pair of static and temporal random vectors and maps them to a set of synthetic latent codes $\hat{h}$. Here, the way to implement function g involves an RNN framework

$$\hat{h}_\sigma = g_\sigma, \quad \hat{h}_t = g_\chi(\hat{h}_\sigma, \hat{h}_{t-1}, z_t) \tag{5}$$

where g_σ and g_χ represent the generators of static and dynamic features, respectively, and z_σ and z_t are sequences sampled from Gaussian distributions and the Winer process within the range of [0, 1].

Discriminators, similar to generators, also begin with the embedding space. The discriminator's function $d : \mathbb{H}_\sigma \times \prod_{t=1}^{T} \mathbb{H}_\chi \to [0, 1] \times \prod_{t=1}^{T}[0, 1]$ is responsible for processing the static and temporal latent representations, outputting classification results $\hat{y}$. Here, d is realized through a bidirectional RNN coupled with a feedforward output layer

$$\hat{y}_\sigma = d_\sigma(\hat{h}_\sigma), \quad \tilde{y}_t = d_\chi(\overleftarrow{\gamma}_t, \vec{\gamma}_t) \tag{6}$$

where $\overleftarrow{\gamma}_t = \overleftarrow{u}_\mathcal{X}(\hat{h}_\sigma, \hat{h}_t, \overleftarrow{\gamma}_{t-1})$ and $\vec{\gamma}_t = \vec{u}_\mathcal{X}(\hat{h}_\sigma, \hat{h}_t, \vec{\gamma}_{t-1})$ represent the backward and forward hidden states of the sequence, respectively. $\overleftarrow{u}_\mathcal{X}$ and $\vec{u}_\mathcal{X}$ represent bidirectional RNN's functions. d_σ and d_χ represent the classification functions in the output layer. $\tilde{y}_\sigma$ and $\tilde{y}_t$ represent the classification

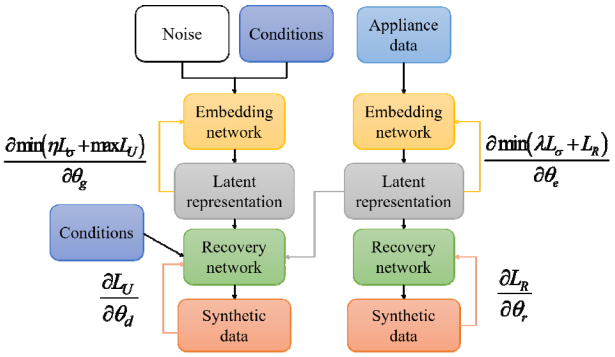

Fig. 2. Framework of C-TimeGAN.

results of either the real y or synthetic $\tilde{y}$ data for static and dynamic features, respectively.

B. C-TimeGAN

To further improve the quality and realism of the generated data, some constraints are added to TimeGAN. Considering that the generated appliances may be correlated and coupled with each other, the power values or the sequences of other appliances are used as constraints when generating data for one appliance. To better learn the characteristics of the rapid/transient changes in the power of appliances, the difference between adjacent time points of the current appliance (first-order difference) is also used as a condition.

Specifically, defining the static appliance as σ^m and the dynamic appliance as χ_t^m, characterizing different appliances when the value of m is changed. Note that all appliances should be included in the dynamic or static data. Therefore, for a static appliance such as fridge, the first-order difference of the fridge can be expressed as c_i^d, where i denotes the number of sampling sets. The correlation of other static appliances except fridge and all the dynamic appliances are used as the constraints c_i^c. Similarly, for dynamic appliances, such as washing machine, the first-order difference of washing machine can be shown as c_j^d, and the other dynamic appliances except washing machine and all the static appliances are used as the constraints c_j^c. c_i^d, c_i^d, c_j^d, and c_j^c are introduced as conditional supervision terms that are used as inputs to the generator along with the random inputs of known distributions. The appliance data containing all dynamic and static features are used as input to the embedding network. The conditions $c_i^{d'}$, $c_i^{d'}$, $c_j^{d'}$, and $c_j^{c'}$, which will be generated, are used as inputs to the discriminator. The framework of the improved C-TimeGAN model is shown in Fig. 2.

Therefore, the embedding network is improved to

$$h_\sigma = e_\sigma\left(\sigma^m, c_j^d, c_j^c\right), \quad h_t = e_\chi\left(h_\sigma, h_{t-1}, \chi_t^m, c_i^d, c_i^c\right). \tag{7}$$

The generator is changed to

$$\hat{h}_\sigma = g_\sigma\left(z_{\sigma^m}, c_j^d, c_j^c\right), \quad \hat{h}_t = g_\chi\left(\hat{h}_\sigma, \hat{h}_{t-1}, z_t, c_i^d, c_i^c\right). \tag{8}$$

The discriminator is changed to

$$\tilde{y}_\sigma = d_\sigma\left(\hat{h}_{\sigma^{m'}}, c_j^{d'}, c_j^{c'}\right), \quad \tilde{y}_t = d_\chi\left(\overleftarrow{\gamma}_t, \vec{\gamma}_t, c_i^{d'}, c_i^{c'}\right). \tag{9}$$

Similar to TimeGAN, the loss function and optimizations of C-TimeGAN can be summarized as follows.

1) *The Reconstruction Loss L_R:* As data transformation between features and latent representations is reversible, the initial loss function aims to assess the precise reconstruction between the embedding and recovery network. The physical meaning of this loss function is to ensure that the embedding and recovery functions can accurately reconstruct $\tilde{\sigma}$, $\tilde{\chi}_{1:T}$ of the original data σ, $\chi_{1:T}$ from their latent representations $h_\sigma, h_{1:T}$

$$L_R = E_{\sigma,\chi_{1:T}\sim p}\left[\| \sigma - \tilde{\sigma} \|_2 + \sum_t \left\| \chi_t - \tilde{\chi}_t \right\|_2\right]. \tag{10}$$

2) *The Unsupervised Loss L_U:*

$$L_U = E_{\sigma,\chi_{1:T}\sim p}\left[\log y_\sigma + \sum_t \log y_t\right] + E_{\sigma,\chi_{1:T}\sim \hat{p}}\left[\log\left(1-\hat{y}_\sigma\right) + \sum_t \log(1-\hat{y}_t)\right] \tag{11}$$

where L_U mirrors the competitive adversarial interaction between the generator and the discriminator. It aims to maximize the likelihood for the discriminator to correctly classify $\hat{y}$ for both the training data $h_\sigma, h_{1:T}$ and synthetic data $\hat{h}_\sigma, \hat{h}_{1:T}$ while minimizing the same for the generator.

3) *The Supervised Loss L_σ:* To model the stepwise dependencies in time series data, we introduce an additional supervised loss based on maximum likelihood to complement binary adversarial learning

$$L_\sigma = E_{\sigma,\chi_{1:T}\sim p}\left[\sum_t \left\| h_t - g_\chi\left(h_\sigma, h_{t-1}, z_t, c_i^d, c_i^c\right)\right\|_2\right] \tag{12}$$

where L_σ measures the similarity between the synthetic data from generator and the real data encoded by the embedding network. This loss measures the discrepancy between the actual and synthetic conditional distributions $p(H_t|H_\sigma, H_{1:t-1})$ and $\hat{p}(H_t|H_\sigma, H_{1:t-1})$, where $g_\chi(h_\sigma, h_{t-1}, z_t)$ approximates $E_{z_t\sim N}[\hat{p}(H_t|H_\sigma, H_{1:t-1}, z_t)]$ with a single sample z_t from the standard normal distribution.

There are two optimization objectives that C-TimeGAN needs to follow during its training:

$$\min_{\theta_e,\theta_r}\left(\lambda L_\sigma + L_R\right) \tag{13}$$

$$\min_{\theta_g}\left(\eta L_\sigma + \max_{\theta_d} L_U\right) \tag{14}$$

where $\theta_e, \theta_r, \theta_g$, and θ_d denote the parameters in embedding, recovery, generation, and discrimination components in C-TimeGAN, respectively, and λ and η are hyperparameters used to balance the losses among L_R, L_U, and L_σ.

For sampling or synthesizing generated data from the trained C-TimeGAN, the model employs the generator $g(g_\sigma, g_\chi)$ to map combinations of static and temporal random vectors into synthetic latent representations $\hat{h}_\sigma, \hat{h}_{1:T}$. It is similar to TimeGAN's sample process in Fig. 1.

The generator network g_σ, which is responsible for static attributes, transforms a random vector z_σ from a selected distribution into a static latent representation $\hat{h}_\sigma$. For the temporal components, g_χ constructs temporal latent representations $\hat{h}_t$ by integrating $\hat{h}_\sigma$, the preceding temporal latent representations $\hat{h}_{t-1}$, and a temporal random vector z_t. z_σ samples from Gaussian distribution and z_t follows the Wiener process.

It is worth noting that, maintaining equal lengths for generated and input data is pivotal in C-TimeGAN. This ensures improved learning of the data's temporal sequence structure. By considering both joint and conditional distributions simultaneously, the model accurately captures time dependencies. As a result, generated sequences exhibit realistic individual time points and consistent overall temporal dynamics akin to real data.

The detailed pseudocode of the innovative C-TimeGAN model is described in the Appendix.

C. Anomaly Detection for Synthetic Data Based on OCSVM

Although C-TimeGAN can generate high-dimensional data, it should also be noted that some synthetic points in the results of the visualization analysis in [12] still deviate from the real 390 data distribution area. To enhance data validity, outlier removal is necessary. This process can be seen as a novelty detection problem, where synthetic data belonging to a given class are determined by using only real data as the input space.

OCSVM is an unsupervised one-class outlier detection model. It uses a nonlinear function to map the training data to a high-dimensional feature space and solves for the spatially optimal classification hyperplane, thereby separating the sample data from the origin of the feature space at maximum intervals. The synthetic data are judged by falling inside or outside the hyperplane, and points outside the hyperplane are considered detected outliers and are rejected [29]. The model is described mathematically as follows [30].

Suppose that the training dataset $P = \{x_1, x_2, x_3, \ldots, x_n\}$ and the testing dataset $Q = \{y_1, y_2, y_3, \ldots, y_n\}$, where n is the number of samples drawn from two identical dimensional distributions of P and Q. $\phi(x)$ is the nonlinear function that maps the samples to high-dimensional space F. The hyperplane expression is $\omega^{\mathrm{T}}x + b = 0$, and then, the optimization objective of the classifier is

$$\begin{aligned} &\min_{w,b,\xi_i} \frac{\| \omega \|^2}{2} + \frac{1}{kn}\sum_{i=1}^{n}\xi_i - b \\ &\text{s.t. } \left(\omega^{\mathrm{T}} \cdot \phi\left(x_i\right)\right) \geq b - \xi_i, \quad i = 1, 2, \ldots, n \\ &\quad \xi_i > 0, \quad i = 1, 2, \ldots, n \end{aligned} \tag{15}$$

where ω and b denote the weights and thresholds of SVM, respectively, and ξ_i denotes the slack variables corresponding to the sample data. The parameter $k \in [0, 1]$ determines the upper limit of the proportion of singularities in the sample and the lower limit of the proportion of support vectors. By utilizing the Lagrange operator and a kernel function, the classification function in (16) can identify outliers in synthetic data

$$\begin{aligned} f(x) &= \operatorname{sgn}\left(\left(\omega^{\mathrm{T}} \cdot \phi\left(x_i\right)\right) - b\right) \\ &= \operatorname{sgn}\left(\sum_{i=1}^{n}\alpha_i K\left(x, x_i\right) - b\right). \end{aligned} \tag{16}$$

D. Computational Complexity Analysis

Suppose that the dimensionality of the noise input equals the dimensionality of the data generation ($Z = D$), with batch size n_{bs} and training epochs e. For TimeGAN, assuming that all components (embedding layer, recovering layer, generator and discriminator, and supervised learning layer) are implemented using an n_{layer} GRU with hidden units U and sliding window length T, then the computational complexity of each component is $O(4 \times T \times U \times D \times n_{\mathrm{layer}} \times n_{\mathrm{bs}} \times e)$. The total complexity consists of three components: 1) embedding network training for embedding and recovering layers $O(2 \times 4 \times T \times U \times D \times n_{\mathrm{layer}} \times n_{\mathrm{bs}} \times e)$; 2) training with supervised loss for generator and supervised layer $O(2 \times 4 \times T \times U \times D \times n_{\mathrm{layer}} \times n_{\mathrm{bs}} \times e)$; and 3) joint training for all components $O(5 \times 4 \times T \times U \times D \times n_{\mathrm{layer}} \times n_{\mathrm{bs}} \times e)$. The whole computation cost is $O(9 \times 4 \times T \times U \times D \times n_{\mathrm{layer}} \times n_{\mathrm{bs}} \times e)$.

For C-TimeGAN, with the addition of constraints of dimensional C to TimeGAN, considering that the constraints are merged with the data to be generated input D, the overall computational complexity will become $O(9 \times 4 \times T \times U \times (D+C) \times n_{\mathrm{layer}} \times n_{\mathrm{bs}} \times e)$. Specifically, the mentioned constraints involve appliance coupling and the first-order difference constraint. In this context, $C = 2$ and the computational complexity approximately increases by a factor of $(C/D) = (2/D)$ compared to TimeGAN.

E. C-TimeGAN$^+$ for NILM

Fig. 3 illustrates the implementation process of generating appliance data and its application based on C-TimeGAN$^+$ (C-TimeGAN$^+$ indicates C-TimeGAN integrated with post-processing of the synthesized data using OCSVM). It mainly includes data preprocessing, network training, anomaly detection, quality assessment, and data augmentation five steps.

1) To generate realistic data, it is necessary to perform preprocessing operations before feeding the data into C-TimeGAN.
   Normalization is a crucial step for appliance data, involving the utilization of corresponding appliance mean and standard deviation values. To achieve actual power values from the synthetic data generated by C-TimeGAN, a denormalization process is necessary. This normalization technique enhances the convergence rate of the generator and discriminator within C-TimeGAN. In our

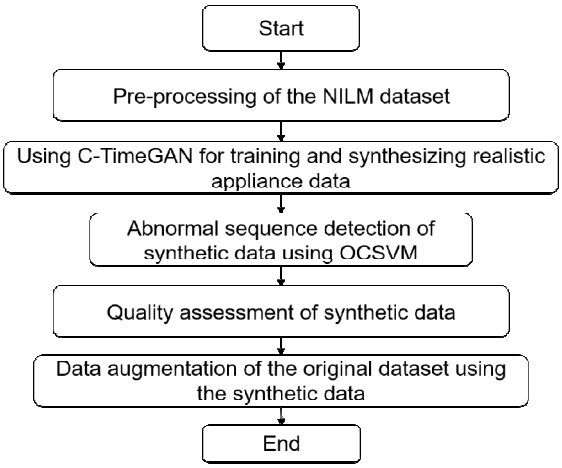

Fig. 3. Flowchart of using C-TimeGAN$^+$ for NILM.

experiments, we opt for setting $\lambda = 1$ and $\eta = 15$ to achieve a balanced loss function.

To facilitate the learning process of individual appliance features, a sliding window methodology is applied for preprocessing the entire sequence. This approach enables C-TimeGAN to effectively grasp the distinctive characteristics of each appliance.

The model adopts a sequence-to-sequence learning strategy, where a sliding window encompassing appliance power data is mapped to the corresponding generated appliance power window.

Following preprocessing, training samples of length L and dimension D (number of appliances) are supplied to C-TimeGAN for data synthesis.

2) To complete the network training and generate realistic appliance data, the training epochs are set and the training appliance data and its dynamic and static feature sequences are input into C-TimeGAN.
3) OCSVM is used to detect and reject outliers in synthetic data to further improve data quality.
4) Quality assessment of synthetic data from a quantitative and qualitative perspective.
5) Data augmentation experiments on the original dataset using synthetic data based on other classical NILM models.

IV. Implementation Details

A. Dataset

Our experiments are conducted in the reference energy disaggregation dataset (REDD) [31], encompassing power consumption measurements obtained from 25 domestic appliances within six residential households situated in USA. The dataset is characterized by sampling frequencies of both 1 and 1/3 Hz. To ensure consistency of sampling, we preprocess the REDD dataset with the procedures outlined in [32]. In order to comprehensively test our generative model, appliances should be selected according to the four typical types defined in [2] and extended in [33]: ON–OFF, finite-state or multistate machine, continuously variable, and always ON. Therefore, six appliances are used to meet the above considerations: fridges, washing machines, lights, air conditioners, dishwashers, and microwave ovens.

TABLE I
Hyperparameter Setting for C-TimeGAN

The main parameters	The values
Sequence length	10000
Number of appliance types	6
Window length	60
Model framework	GRU
Hidden units	24
Hidden layers	3
Batch size	128
Epochs	2000
Decay rate	0.1
Decay steps	1000
Optimizer	Adam
Learning rate	0.001
$\beta_1, \beta_2, \hat{o}$	$0.9, 0.999, 10^{-8}$
Dropout	0.1
λ, η	1,15

B. Hyperparameters and Benchmarks

Our experiments are conducted using TensorFlow and Keras within a Python 3 environment. The training parameters for our C-TimeGAN model are detailed in Table I. The training is equipped on a machine with a NVIDIA H100 GPU.

In the training of the C-TimeGAN model, synthetic sequences for six distinct appliance types are chosen to generate, each with a sequence length of 10 000. A sliding window of size 60 is employed to capture temporal dependencies. The architecture is constructed using three-layer gated recurrent unit (GRU) networks for embedding, recovery, and the generative and discriminative components, with the hidden layers dimensioned at four times the number of appliance types. Data normalization is applied to facilitate model convergence, and the generated sequences undergo inverse normalization for result interpretation, with λ and η set to 1 and 15, respectively.

GAN [19], DCGAN [27], and TimeGAN [12] are used as baselines for comparison with C-TimeGAN. All the trainings employ the Adam optimizer for refinement, setting at a learning rate of 0.0001 with beta values of 0.9 and 0.999. To mitigate the risk of overfitting, a learning rate decay strategy and dropout are incorporated into these models. Each model is trained for 2000 epochs with each 1000 epochs, including decaying with inverse time weights.

C. Evaluation Metrics

In contrast to other tasks within NILM that involve clearly defined and quantifiable targets, it remains a challenge to establish metrics for assessing GAN models [34]. The metrics selected for C-TimeGAN$^+$, a variant of GAN, including Fréchet inception distance (FID) [35], sliced Wasserstein distance (SWD) [36], classification score, prediction score, and visualization methods, including t-distributed stochastic neighbor embedding (t-SNE) [37] and principal component

TABLE II
DETAILED DESCRIPTION OF EVALUATION METRICS

Metrics	Description	Goal	Unit
FID	Compares distributions of generated and real data via Inception V3's output features	Assesses quality and distribution closeness; lower means better	Dimensionless
SWD	Measures dissimilarities between signal distributions, aligning with methodologies from [8]	Similarity measurement between synthetic and real data; lower means closer similarity	Dimensionless
Classification Score	Utilizes a GRU-based model to distinguish between synthetic and real data, reporting classification error	Evaluates fidelity; lower score indicates better mimicry	Dimensionless
Prediction Score	Trains a GRU model on synthetic data, evaluating mean absolute error (MAE) against real data	Quantifies usefulness; lower MAE means higher usefulness	Watts
Visualization Methods (t-SNE and PCA)	Maps data to lower dimensions for qualitative comparison of distributions	Qualitatively assesses data diversity	Dimensionless

analysis (PCA) [38], are meticulously adapted to evaluate the quality of synthetic power data. The detailed description of these metrics is shown in Table II.

Each metric is specifically adapted to gauge three critical aspects of synthetic data.

1) *Fidelity:* The synthetic data should be indistinguishable from the real one.
2) *Usefulness:* Training on synthetic data and testing on real data.
3) *Diversity:* The distribution of the synthetic data should cover the real data.

V. EXPERIMENTS

To improve the accuracy of synthetic data, we have incorporated two post hoc mechanisms, as described in [8], into our generation process. The first mechanism ensures that the synthetic power at any time step is greater than zero. The second mechanism involves dropping any synthetic samples that do not meet the energy threshold and replacing them with newly generated synthetic data. These simple but effective mechanisms enhance the fidelity of synthetic data and promote its utility in real-world applications.

A. Convergence Analysis on Different GAN Models

Performing a convergence analysis on C-TimeGAN is essential, as it highlights the model's superior stability and reliability over benchmarks.

Fig. 4 presents the convergence results of four GAN models obtained after 2000 training epochs. In Fig. 4(a), it is observed that the original GAN encounters mode collapse around 1200 epochs, evident as a sudden decrease in generator loss and an increase in discriminator loss. This reduction does not signify performance improvement; rather, it signifies the discovery of a method to "deceive" the discriminator by repetitively generating specific patterns.

For the DCGAN, toward the latter part of training, the generator loss gradually increases, while the discriminator loss remains relatively low, suggesting that the discriminator has become too powerful, causing a stagnation in the generator's learning and a lack of diverse adjustments.

In TimeGAN, both generator and discriminator losses consistently decrease amid fluctuations, indicating continuous learning and effectively evading mode collapse issues.

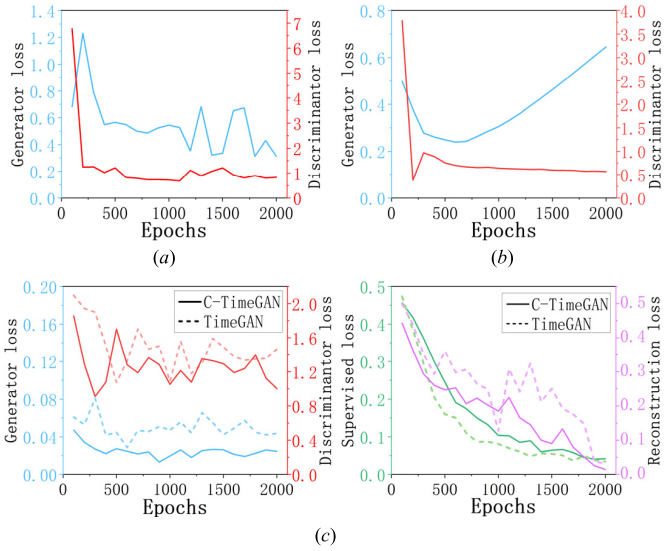

Fig. 4. Convergence analysis of different GAN models. All losses are dimensionless. (a) GAN. (b) DCGAN. (c) TimeGAN (dashed line) and C-TimeGAN (solid line).

Compared to TimeGAN, C-TimeGAN inherits the ability to solve mode collapse and demonstrates faster convergence with lower loss values across the generating, discriminating, and reconstruction network. Around 400 epochs, the generator converges to an acceptable level of loss, and the network loss continues to improve, enhancing the model's performance. Despite a slight increase in supervised loss, this is attributable to the model's generation process needing to satisfy additional constraint conditions, ensuring the generated data embodies more dynamic characteristics.

Table III presents the time consumption for each epoch during the training of various models. It can be observed that the time required for GAN is the least. In comparison to TimeGAN, our C-TimeGAN model exhibits an increase in time by approximately 36.98%, remaining within the same order of magnitude. This aligns with the theoretically calculated 33.33% (the generated dimension D is set to 6 in experiments).

B. Quantitative Results

Table IV presents the results of various metrics, including FID, SWD, classification score, and prediction score, for

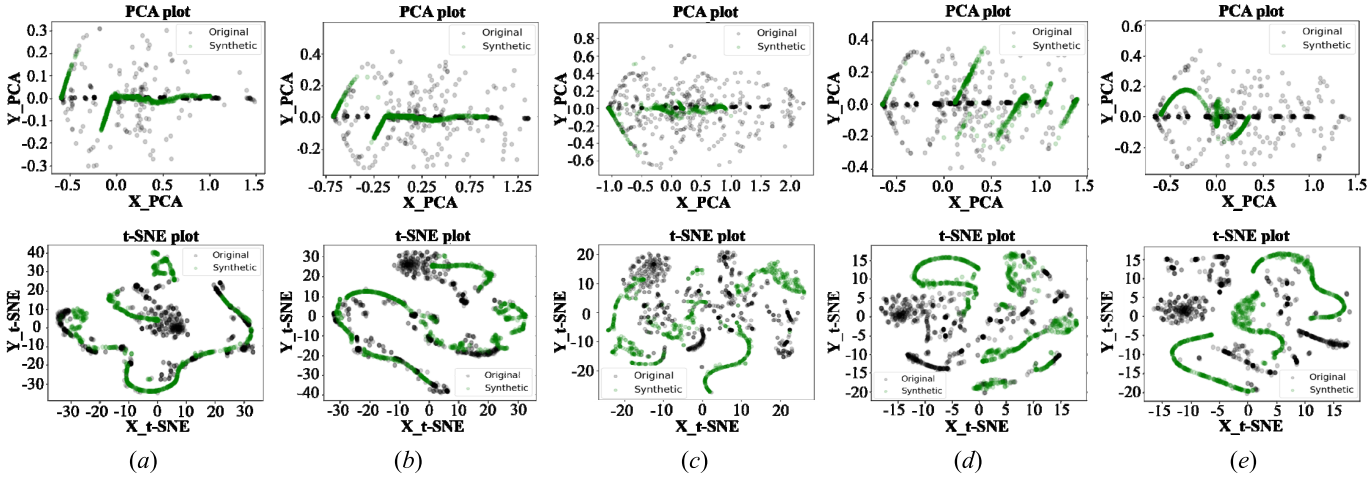

Fig. 5. Visualization results of the original and synthetic appliance power data from GAN, DCGAN, TimeGAN, C-TimeGAN, and C-TimeGAN$^+$ by PCA (1st row) and t-SNE (2nd row). Black indicates original data and green indicates synthetic data. (a) C-TimeGAN$^+$. (b) C-TimeGAN. (c) TimeGAN. (d) DCGAN. €GAN.

TABLE III
TRAINING TIME COMPARISON ACROSS DIFFERENT MODELS

Model	GAN	DCGAN	TimeGAN	C-TimeGAN
Time (s) / one epoch	0.1219	0.1495	0.9572	1.3112

TABLE IV
METRICS FOR EVALUATING SYNTHESIZED ALL APPLIANCE DATA USING VARIOUS MODELS (ORIGIN MEANS THE RESULTS OF ORIGINAL REDD DATA IN THESE EVALUATION METRICS)

Method	FID	SWD	Classification score	Prediction Score
Origin	0	0	0	0.122±0.001
GAN	70.49	0.61±0.097	0.656±0.018	0.509±0.007
DCGAN	57.86	0.37±0.058	0.484±0.011	0.278±0.004
TimeGAN	40.39	0.18±0.032	0.284±0.014	0.133±0.003
C-TimeGAN	34.22	0.11±0.037	0.236±0.013	0.128±0.003
C-TimeGAN$^+$	**33.65**	**0.10±0.032**	**0.217±0.012**	**0.124±0.003**

different models generating multiple appliance trajectories simultaneously. Among the GAN frameworks evaluated, our C-TimeGAN$^+$ model exhibits the best performance, with a 71.9% and 109.4% improvement in FID and 270% and 510% improvement in SWD over DCGAN and GAN models, respectively. This indicates that our model can generate realistic data better than the baseline models. For the classification score and prediction score metrics, our improved model outperforms the DCGAN model by 76.2% and 85.8%, respectively. This scenario indicates that our model can generate higher quality synthetic appliance trajectories that are indistinguishable from the original data (shown in the classification score) and retain predictive features (shown in the prediction score). By comparing TimeGAN and C-TimeGAN, it can be seen that the performance of TimeGAN can be enhanced in various metrics by adding constraints. The results also suggest that OCSVM-based anomaly detection can further improve the performance of C-TimeGAN. Notably, our model's prediction score is almost on par with the score of the original data itself.

Table V shows the results of the same evaluation metrics for different models generating only a single appliance trajectory at a time. It can be seen that for different appliances, our model can still consistently produce higher quality trajectories, and all metrics are better than those obtained when multiple appliance trajectories are generated at once. Compared to TimeGAN, our C-TimeGAN$^+$ model exhibits notable improvements in FID metrics across fridge, microwave oven, dishwasher, washing machine, light, and air conditioner by 30.1%, 58.6%, 35.4%, 29.8%, 20.8%, and 15.3%, respectively. These enhancements beyond TimeGAN validate the effectiveness of the additional constraints introduced. Remarkably, the prediction score of our model for generating data for multiple appliances at once is almost identical to the prediction score for generating data for a single appliance at once (0.124 versus 0.121 W). This finding confirms that our model can generate multidimensional data for prediction purposes in a single iteration, thereby improving synthetic efficiency and practical significance. In general, the prediction score of the original data is better than those of the generated data. However, this is not the case for the fridge data generated by C-TimeGAN$^+$. This is because the fridge data from C-TimeGAN$^+$ contain less noise than the original fridge data (there are some abrupt noises due to the sensor errors), which is more conducive to the prediction task.

C. Qualitative Results

To ensure accurate quantitative evaluation of generated signals, it is crucial to consider the realism, diversity, and novelty of appliance trajectories. The diversity of appliance trajectories is observed both between different appliances and within individual appliances. To address these aspects, we will employ visualization methods for realism, overall analysis for novelty, and detailed analysis for the diversity of appliance trajectories in our quantitative results.

Fig. 5 presents the visualization outcomes of the generated and original power data from various models using PCA and t-SNE. It is evident that the appliance power data produced by C-TimeGAN$^+$ in both PCA and t-SNE

TABLE V
METRICS FOR EVALUATING SYNTHESIZED SINGLE APPLIANCE DATA USING VARIOUS MODELS. BOLDED DENOTES BEST RESULTS

Metric	Method	Fridge	Microwave oven	Dishwasher	Washing machine	Light	Air conditioner	Average
FID	GAN	48.256	45.164	98.327	103.861	22.906	38.463	59.50±33.43
	DCGAN	35.825	36.909	57.963	85.299	13.137	26.158	42.55±25.60
	TimeGAN	9.755	12.907	28.649	29.227	6.878	17.546	17.49±13.23
	C-TimeGAN	7.407	6.215	22.036	24.795	5.951	16.145	13.75±9.28
	C-TimeGAN$^+$	**6.814**	**5.342**	**18.504**	**20.808**	**5.513**	**14.862**	**11.97±6.95**
SWD	GAN	0.324±0.021	0.385±0.017	0.460±0.053	0.537±0.039	0.104±0.0081	0.372±0.027	0.363±0.027
	DCGAN	0.165±0.0041	0.172±0.0086	0.229±0.044	0.463±0.029	0.048±0.0017	0.216±0.0083	0.215±0.016
	TimeGAN	0.073±0.0046	0.105±0.0063	0.166±0.041	0.230±0.020	0.035±0.0025	0.119±0.0046	0.121±0.013
	C-TimeGAN	0.061±0.0057	0.086±0.0050	0.149±0.035	0.183±0.021	0.033±0.0028	0.095±0.0038	0.101±0.012
	C-TimeGAN$^+$	**0.059±0.0055**	**0.083±0.0047**	**0.144±0.028**	**0.178±0.017**	**0.031±0.0026**	**0.086±0.0034**	**0.097±0.010**
Classification score	GAN	0.498±0.012	0.397±0.006	0.272±0.008	0.236±0.006	0.106±0.005	0.210±0.005	0.287±0.007
	DCGAN	0.249±0.009	0.191±0.005	0.213±0.010	0.227±0.005	0.042±0.002	0.153±0.002	0.191±0.006
	TimeGAN	0.148±0.006	0.052±0.003	0.046±0.004	0.081±0.004	0.010±0.000	0.084±0.002	0.070±0.003
	C-TimeGAN	0.129±0.006	0.036±0.002	0.022±0.002	0.058±0.004	0.006±0.000	0.076±0.002	0.054±0.002
	C-TimeGAN$^+$	**0.114±0.005**	**0.034±0.002**	**0.019±0.001**	**0.050±0.003**	**0.004±0.000**	**0.063±0.002**	**0.047±0.002**
Prediction score	GAN	0.501±0.005	0.361±0.003	0.313±0.003	0.307±0.002	0.245±0.002	0.434±0.004	0.360±0.003
	DCGAN	0.323±0.005	0.245±0.002	0.262±0.003	0.175±0.003	0.084±0.001	0.301±0.004	0.231±0.003
	TimeGAN	0.243±0.004	0.110±0.002	0.102±0.002	0.092±0.002	0.038±0.001	0.209±0.004	0.132±0.003
	C-TimeGAN	0.236±0.004	0.102±0.002	0.094±0.002	0.084±0.002	0.033±0.001	0.201±0.004	0.125±0.003
	C-TimeGAN$^+$	**0.230±0.004**	**0.096±0.002**	**0.091±0.002**	**0.081±0.002**	**0.032±0.001**	**0.197±0.004**	**0.121±0.003**
	Origin	0.233±0.003	0.087±0.002	0.086±0.002	0.077±0.002	0.028±0.001	0.175±0.003	0.114±0.002

visualization plots can exhibit a substantial overlap with the original data, unlike the baseline. This indicates that our synthetic data distribution can better encompass genuine data, showcasing features of both fidelity and diversity. By contrasting TimeGAN with C-TimeGAN, it is apparent that the performance of the C-TimeGAN model significantly outshines TimeGAN in generating NILM data. The performance boost primarily stems from the integration of additional constraints within C-TimeGAN. To be specific, appliance-related coupling constraints and first-order difference constraints in power dynamics significantly drive the performance improvement, aiding the model in capturing the nuanced and rapid power changes in data dynamics for a more accurate representation. By comparing the results of C-TimeGAN and C-TimeGAN$^+$ in the first two columns, after using OCSVM to remove anomalies from the synthetic data, further improvements can be observed in the visualizations. Specifically, for the PCA plots, C-TimeGAN$^+$ eliminates a large number of outliers in the generated data and corresponding original data, where the Y_PCA components are greater than 0.3, making the synthetic data cleaner. For the t-SNE plots, the generated data from C-TimeGAN$^+$ align better with the original data, with fewer offset data points. This finding further emphasizes the effectiveness of the anomaly detection mechanism.

Fig. 6 provides an overview of C-TimeGAN$^+$-generated and original appliance trajectories for six different appliances. Overall, it is apparent that the generated signals do not exactly replicate the original signals, exhibiting variations in power values, operating periods, activation frequencies, and other features. However, the generated signals closely resemble the original signals and capture all the main characteristics of each appliance class. In the following, we provide a detailed analysis of each appliance's distinctive features.

Fridge: The synthetic fridge signal exhibits the original periodic characteristics and the overshoot state when switching ON, while the ON/OFF frequency varies from the real data and changes over time, leading to a substantial increase in the diversity of fridge data.

Microwave Oven: The synthetic microwave oven trajectories accurately portray the high-power consumption values, low activation frequency, and post-state of real microwave ovens signal.

Dishwasher: The generated dishwasher trajectory can fit the multistate dishwasher processes, including instantaneous overshoot at the opening, post-state at the end, smooth operation in high-power states, and quick fluctuations and gaps in low-power state operation.

Washing Machine: Synthetic washing machine signals try to convey the complex and various states of the washing machine. It retains the overshoot state and the fast switching ON/OFF state at work.

Light: The synthesis process of the light signal is relatively easy as it only has two processes, ON and OFF. The synthetic light signal preserves the "half-circle waveform" when switched ON and the fluctuations during operation (which are caused by sensors or unknown load noise).

Air Conditioner: The synthesis process of air conditioner is very similar to that of fridge in that both are periodic. However, the air conditioner is a little more complex because it is not activated at a fixed frequency and also varies over time. The synthetic air conditioner signal keeps this characteristic.

In order to further quantitatively evaluate the diversity of power trajectories generated by C-TimeGAN$^+$, two detailed examples of synthetic and real fridge signals are presented in Fig. 7. The synthetic trajectories not only retain typical features of fridges, such as transient spikes, working fluctuation, and post-state, but also exhibit variations in power value,

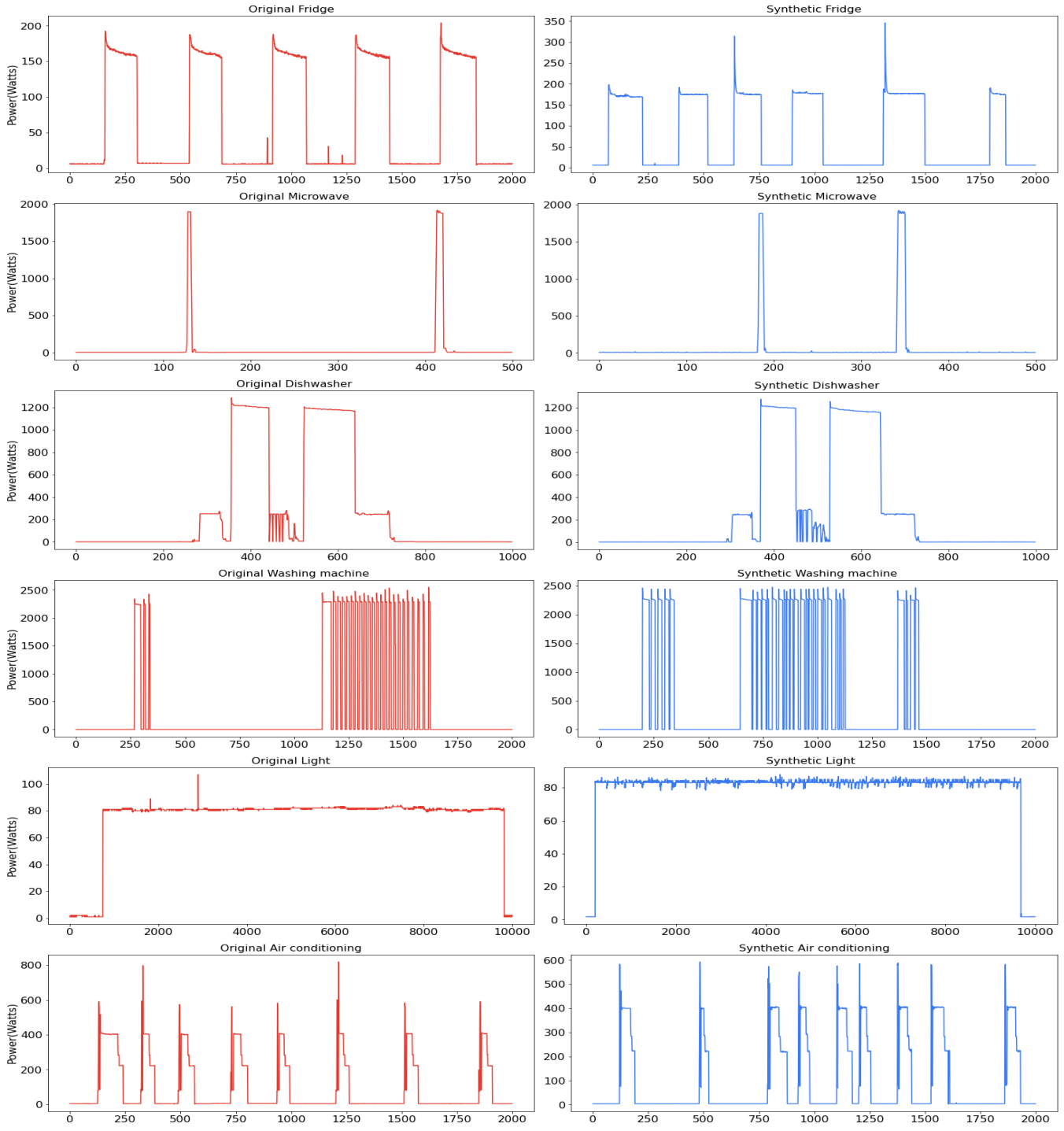

Fig. 6. Overall examples of original and synthesized appliance trajectories by using C-TimeGAN^{+}. Units are in watts.

ON/OFF frequency, and operating period over time, indicating high diversity.

D. Hyperparameters Sensitivity Analysis

Performing further sensitivity analyses would be beneficial to assess the C-TimeGAN's generalization and stability across diverse appliance data and environmental conditions. Specifically, we conduct a series of experiments on two key hyperparameters, $\lambda = \{1, 5, 10, 15, 20\}$ and $\eta = \{0.1, 0.5, 1, 2, 5\}$, which directly influence the model's optimization objectives. These analyses are focused on data generation for fridges (single-state) and washing machines (multistate). The detailed results are averaged from five training times and can be observed in Table VI.

From Table VI, it is observed that the FID metric related to the synthetic fridge data has minimum and maximum values of 7.407 and 8.978, respectively, with a variance of 0.296. In addition, the prediction score metric for the

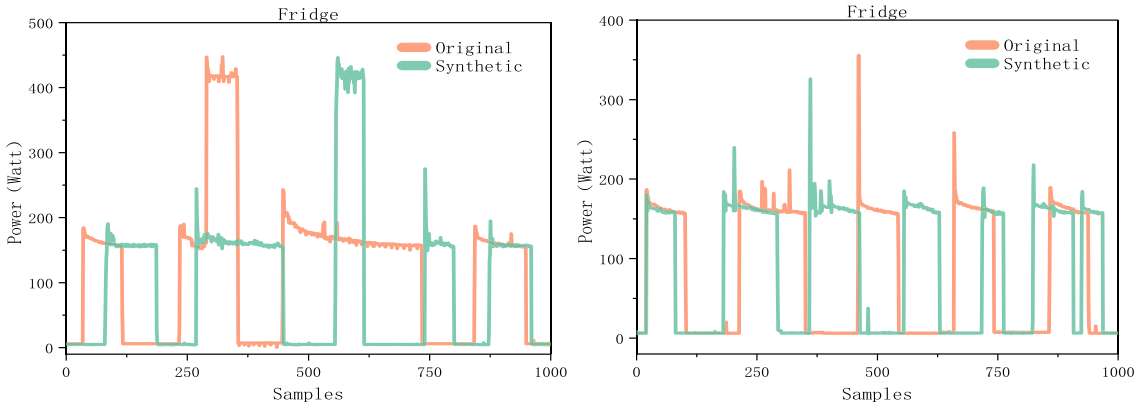

Fig. 7. Detailed examples of original and synthesized fridge appliance trajectories by using C-TimeGAN$^+$. Units are watts.

TABLE VI

SENSITIVITY ANALYSIS OF C-TIMEGAN HYPERPARAMETERS λ AND η ON FRIDGE AND WASHING MACHINE DATA GENERATION PERFORMANCE. BOLDED DENOTES BEST RESULTS

Appliance	λ	η	FID	SWD	Classification score	Prediction Score
Fridge	15	0.1	8.249	0.067±0.0058	0.136±0.007	0.243±0.004
		0.5	8.978	0.066±0.0059	0.140±0.007	0.241±0.004
		2	7.431	**0.058±0.0055**	0.133±0.006	0.244±0.004
		5	7.754	0.069±0.0061	0.151±0.007	0.246±0.004
	1	1	8.606	0.067±0.0062	0.147±0.007	0.237±0.004
	5		8.172	0.066±0.0058	0.132±0.006	0.238±0.004
	10		7.599	0.062±0.0057	**0.123±0.006**	0.248±0.004
	20		7.823	0.066±0.0058	0.131±0.006	0.245±0.004
	Baseline $\lambda=15,\eta=1$		**7.407**	0.061±0.0057	0.129±0.006	**0.236±0.004**
	Average		7.970±0.296	0.065±0.0058	0.135±0.006	0.241±0.004
Washing machine	15	0.1	25.861	0.185±0.020	0.062±0.004	0.092±0.002
		0.5	26.137	0.188±0.021	0.067±0.004	**0.081±0.002**
		2	24.893	**0.179±0.020**	0.064±0.004	0.088±0.002
		5	27.472	0.202±0.022	0.075±0.004	0.090±0.002
	1	1	26.496	0.192±0.022	0.073±0.004	0.087±0.002
	5		24.981	0.185±0.020	0.064±0.004	0.082±0.002
	10		25.374	0.186±0.020	0.063±0.004	0.086±0.002
	20		25.010	0.187±0.021	0.059±0.004	0.085±0.002
	Baseline $\lambda=15,\eta=1$		**24.795**	0.183±0.021	**0.058±0.004**	0.084±0.002
	Average		25.606±0.804	0.187±0.021	0.064±0.004	0.085±0.002

washing machine generated data ranges from a minimum of 0.081 W to a maximum of 0.092 W, with a variance of 0.002 W. These results indicate a moderate sensitivity of our C-TimeGAN model to the parameters λ and η, demonstrating robust performance. This resilience to slight variations in parameters ensures consistent output quality, enhances the model's reliability in diverse scenarios, and reduces the need for precise parameter tuning, thereby simplifying the model's practical application and deployment.

E. Data Augmentation for NILM

The synthetic data produced by our model can be utilized as a practical approach for data augmentation to enrich the training samples and thereby enhance the monitoring accuracy and generalization of the disaggregation model. To demonstrate this, we conducted data augmentation experiments on the REDD dataset using a simple long short-term memory (LSTM) model [39]. We opt to use the LSTM model instead of the TimeGAN-variant model for the NILM task to avoid the synthetic data potentially benefiting from the disaggregation of the latter model.

To conduct the experiments, baselines are first established by training LSTMs individually for each of the six appliances in the REDD dataset, including all appliance classes generated by our model. To ensure consistency, the experiments are conducted in a single house, with the fridge, microwave oven, washing machine, dishwasher, and light training in House 1 and the air conditioner trained in House 6. Each appliance is trained separately for 250 epochs, using the Adam optimizer with a learning rate of 0.001 and betas of 0.9 and 0.999. The training and testing sets are split in a 9:1 ratio, and the best results are saved for evaluation.

The augmentation experiments are performed using C-TimeGAN$^+$-generated data to randomly replace $n\%$ of the ground truth appliance data on the REDD dataset. It is worth noting that: 1) the synthetic samples are only involved in training and not in testing, and the testing set can only be selected from real samples; 2) the appliance power is used as the model label during training, and all operations during

TABLE VII
RESULTS OF DIFFERENT EXPERIMENTS USING C-TIMEGAN+ FOR DATA AUGMENTATION ON THE REDD DATASET

Experiments	Fridge			Microwave oven			Dishwasher		
	MSE	MAE	F1	MSE	MAE	F1	MSE	MAE	F1
Baseline	3322.50	23.25	90.62	18343.30	21.69	55.11	11355.35	18.87	41.13
1% augmentation	3155.14	22.47	91.53	17972.65	19.47	56.79	**10025.83**	**17.65**	**43.62**
2% augmentation	**2998.28**	21.22	91.74	17745.08	18.50	57.93	11652.41	18.95	42.94
5% augmentation	3032.10	**21.06**	**91.82**	**17566.12**	**18.22**	**58.56**	10893.76	19.21	42.17
100% augmentation	3419.07	23.83	90.71	20532.34	23.74	51.75	13043.64	21.52	35.41

Experiments	Washing machine			Light			Air conditioner		
	MSE	MAE	F1	MSE	MAE	F1	MSE	MAE	F1
Baseline	42863.41	38.97	48.03	2043.52	20.21	75.38	3681.08	19.48	58.23
1% augmentation	41938.53	**37.74**	**49.69**	2054.60	20.13	75.81	3716.64	18.24	59.64
2% augmentation	**40695.39**	38.44	47.19	1790.39	18.46	76.35	3426.51	**18.12**	60.72
5% augmentation	43157.83	38.75	46.52	**1573.12**	**17.32**	**77.24**	**3360.57**	18.29	**61.51**
100% augmentation	48051.38	43.53	42.26	2124.63	21.55	73.79	3882.15	20.33	55.59

data augmentation are not only for the appliance power, but the total power needs to be added the difference between the synthetic appliance power and the original appliance power at the replacement time point; and 3) we give preference to random replacement for data augmentation using the activated portion of the synthetic appliance power. N is a relatively small value because some appliances are only active for a small proportion of the time (e.g., washing machines and dishwashers are used less than 10% of the total time).

Remarkably, due to the relative richness of the augmented training set, models trained using the augmented dataset reach their respective best performance sooner. Therefore, the early stopping mechanism is added to prevent overfitting. The mean squared error (MSE, unit is the square of watts); mean absolute error (MAE, unit is watts), and F1 scores (dimensionless) are used to evaluate the performance of the data augmentation experiments.

Table VII shows the results of different experiments using TimeGAN for data augmentation on the REDD dataset. It is noticed that with a 1% data augmentation, all appliances show a performance improvement compared to the baseline. For example, in terms of MAE metrics, fridge, microwave oven, dishwasher, washing machine, light, and air conditioner are reduced by 3.3%, 10.2%, 6.5%, 3.2%, 0.4%, and 6.3%, respectively. This can be explained by two advantages of data augmentation of our model: 1) enriching the diversity of training samples and thus improving the generalization of the LSTM model and 2) increasing the robustness of the LSTM model by adding noise data (noise always exists in the activation of each appliance). In other words, adding the data generated by our model can actually be seen as a means of regularization, which further prevents the LSTM model from being overfitting during training.

However, in the cases of 2% data augmentation, the performance of the low activation times appliance (dishwasher and washing machine) becomes worse compared to the cases of 1% data augmentation or even the baseline, although further improvement for other finite-state appliances. The same scenario occurs for 5% data augmentation. This is because overaugmentation of the data is equivalent to over-regularization, which may result in the LSTM model not fully learning the multistate characteristics of the appliance and remaining in an underfitting state. However, further increases of n are beneficial for finite-state appliance models that still need to be protected against overfitting. Therefore, it is possible to further improve the performance of model by selecting the best augmentation parameters for each appliance separately.

TABLE VIII
OPTIMAL DATA AUGMENTATION RATIO n AND THE PROPORTION OF ACTIVATION TIME IN THE SAMPLE FOR EACH APPLIANCE

Appliance	Best n	Activation rate	Best n : Activation rate
Fridge	15%	50%	1:3.3
Microwave	5%	15%	1:3
Dishwasher	1%	4%	1:4
Washing machine	1%	3%	1:3
Light	10%	32%	1:3.2
Air conditioner	10%	35%	1:3.5

It is worth mentioning that for 100% data augmentation, which means training with all synthetic data, although the results are not as good as the previous data augmentation experiments, the maximum drop in various metrics is no more than 15%. This indicates that the synthetic data are still valid and the results are consistent with the prediction score from the quantitative analysis.

To further explore the optimal data augmentation n for each appliance, additional experiments are conducted for every 5% increase in n. For a better assessment of the impact of data augmentation on the appliance, we introduce a comprehensive metric C in (17). The best result and the proportion of time the appliance is activated are listed in Table VIII. It can be concluded that the comprehensive performance C of every appliance is best when the ratio of data augmentation n to the activation rate is between 1:3 and 1:4

$$C = \frac{\text{MSE}+\text{MAE}}{\text{MSE}+\text{MAE}+(1-F1)}(1-F1) + \frac{\text{MSE}+(1-F1)}{\text{MSE}+\text{MAE}+(1-F1)}\text{MAE} + \frac{\text{MAE}+(1-F1)}{\text{MSE}+\text{MAE}+(1-F1)}\text{MSE}. \quad (17)$$

Algorithm 1 Pseudocode of C-TimeGAN

Input: $\lambda = 1, \eta = 15$, training dataset $\mathcal{T} = \{(\sigma_n, h_{n,1:T_n})\}_{n=1}^{N}$, total number of samples N, total number of appliance types N_t, constraints $c_j^d, c_j^c, c_i^d, c_i^c$, batch size n_{bs}, learning rate $l_r = 0.001, \epsilon = 10^{-8}, \beta_1 = 0.9, \beta_2 = 0.999$
Initialize: $\theta_e, \theta_r, \theta_g, \theta_d$
while not converged **do**
  (1) Map between input features and latent representations
  Sample $(\sigma_1, h_{1,1:T_n}), \ldots, (\sigma_{n_{bs}}, h_{n_{bs},1:T_{n_{bs}}}) \overset{i.i.d.}{\sim} \mathcal{T}$
  for $n = 1, \ldots, n_{bs}, t = 1, \ldots, T_n$ **do**
    $(h_{n,\sigma}, h_{n,t}) = (e_\sigma(\sigma_n^m, c_j^d, c_j^c), e_\chi(h_{n,\sigma}, h_{n,t-1}, \chi_{n,t}^m, c_i^d, c_i^c))$
    $(\tilde{\sigma}_n, \tilde{\chi}_{n,t}) == (r_\sigma(h_{n,\sigma}), r_\chi(h_{n,t}))$

  (2) Generate synthetic latent representations
  Sample $(z_{\sigma,1}, z_{1,1:T_n}), \ldots, (z_{\sigma,n_{bs}}, z_{n_{bs},1:T_{n_{bs}}}) \overset{\text{i.i.d.}}{\sim} p_{\mathbb{Z}_{\sigma\times\chi}}$
  for $n = 1, \ldots, n_{bs}, t = 1, \ldots, T_n$ **do**
    $(\hat{h}_{n,\sigma}, \hat{h}_{n,t}) = (g_\sigma(z_{\sigma^m,n}, c_j^d, c_j^c), g_\chi(\hat{h}_{n,\sigma^m}, \hat{h}_{n,t-1}, z_{n,t}, c_i^d, c_i^c))$

  (3) Distinguish between real and synthetic latent representations
  for $n = 1, \ldots, n_{bs}, t = 1, \ldots, T_n$ **do**
    $(y_{n,\sigma}, y_{n,t}) = (d_\sigma(h_{n,\sigma}, c_j^{d'}, c_j^{c'}), d_\chi(\overleftarrow{\gamma}_t, \vec{\gamma}_t, c_i^{d'}, c_i^{c'}))$
    $(\hat{y}_{n,\sigma}, \hat{y}_{n,t}) = (d_\sigma(\hat{h}_{n,\sigma}, c_j^{d'}, c_j^{c'}), d_\chi(\hat{\overleftarrow{\gamma}}_t, \hat{\vec{\gamma}}_t, c_i^{d'}, c_i^{c'}))$

  (4) Compute reconstruction loss $\hat{L}_R$, unsupervised loss $\hat{L}_U$ and supervised loss $\hat{L}_\sigma$
  $\hat{L}_R = \frac{1}{n_{bs}} \sum_{n=1}^{n_{bs}} \left[\| \sigma_n - \tilde{\sigma}_n \|_2 + \sum_t \left\| \chi_{n,t} - \tilde{\chi}_{n,t} \right\|_2 \right]$
  $\hat{L}_U = \frac{1}{n_{bs}} \sum_{n=1}^{n_{bs}} \left[\left[\log y_{n,\sigma} + \sum_t \log y_{n,t} \right] + \left[\log\left(1 - \hat{y}_{n,\sigma}\right) + \sum_t \log\left(1 - \hat{y}_{n,t}\right) \right] \right]$
  $\hat{L}_\sigma = \frac{1}{n_{bs}} \sum_{n=1}^{n_{bs}} \left[\sum_t \left\| h_{n,t} - g_\chi\left(h_{n,\sigma}, h_{n,t-1}, z_{n,t}, c_i^d, c_i^c\right) \right\|_2 \right]$

  (5) Update $\theta_e, \theta_r, \theta_g, \theta_d$ via adaptive moment estimation (Adam)
  $\theta_e = \theta_e - \frac{l_r}{\sqrt{\hat{v}_{\theta_e,t}} + \epsilon} \hat{m}_{\theta_e,t} - [\lambda \hat{L}_\sigma + \hat{L}_R]$
  $\theta_r = \theta_r - \frac{l_r}{\sqrt{\hat{v}_{\theta_r,t}} + \epsilon} \hat{m}_{\theta_r,t} - [\lambda \hat{L}_\sigma + \hat{L}_R]$
  $\theta_g = \theta_g - \frac{l_r}{\sqrt{\hat{v}_{\theta_g,t}} + \epsilon} \hat{m}_{\theta_r,t} - [\eta \hat{L}_\sigma + \hat{L}_U]$
  $\theta_d = \theta_d + \frac{l_r}{\sqrt{\hat{v}_{\theta_d,t}} + \epsilon} \hat{m}_{\theta_r,t} - \hat{L}_U$
  where $\hat{m}_{\theta,t} = \frac{m_t}{1-\beta_1^t} = \frac{\beta_1 \cdot m_{t-1} + (1-\beta_1)\cdot(\nabla_\theta)}{1-\beta_1^t}$, $\hat{v}_{\theta,t} = \frac{v_t}{1-\beta_2^t} = \frac{\beta_2 \cdot v_{t-1} + (1-\beta_2)\cdot(\nabla_\theta)^2}{1-\beta_2^t}$, $\theta = \theta_e, \theta_r, \theta_g, \theta_d$
(6) Generate synthetic data
(6-1) Sample $(z_{\sigma,1}, z_{1,1:T_n}), \ldots, (z_{\sigma,N}, z_{N,1:T_N}) \overset{\text{i.i.d.}}{\sim} p_{\mathbb{Z}_{\sigma\times\chi}}$
(6-2) Generate synthetic latent representations
for $n = 1, \ldots, N, t = 1, \ldots, T_n$ **do**
  $(\hat{h}_{n,\sigma}, \hat{h}_{n,t}) = (g_\sigma(z_{\sigma^m,n}, c_j^d, c_j^c), g_\chi(\hat{h}_{n,\sigma^m}, \hat{h}_{n,t-1}, z_{n,t}, c_i^d, c_i^c))$
(6-3) Map back to the original feature space
for $n = 1, \ldots, N, t = 1, \ldots, T_n$ **do**
  $(\hat{\sigma}_n, \hat{\chi}_{1:T_n}) == (r_\sigma(h_{n,\sigma}), r_\chi(h_{n,t}))$
Output: $\hat{\mathcal{T}} = \{(\hat{\sigma}_n, \hat{h}_{n,1:T_n})\}_{n=1}^{N}$

VI. Conclusion

In this article, we proposed a novel model named C-TimeGAN that builds upon TimeGAN by incorporating static and dynamic features of input data as well as differences between adjacent time points as constraints, to generate high-quality and realistic appliance data in the field of NILM. To further enhance the quality of synthetic data, we introduced OCSVM as a postprocessing method to detect anomalous sequences and remove outliers. We have demonstrated that our model outperforms the baseline from a quantitative perspective, such as a 71.9% improvement in FID and a 210% improvement in SWD compared to DCGAN. Besides, we have also proved the fidelity, diversity, and novelty of our synthetic data in qualitative aspects. Furthermore, we have shown that C-TimeGAN$^+$ can act as a practical data augmentation tool for improving the disaggregation accuracy and generalization ability of other classical NILM models. Through further experiments, we have found that the comprehensive performance of each appliance is optimal when the ratio between the data augmentation n and the activation rate is between 1:3

and 1:4, thus enhancing the practical utility of synthetic data.

The limitations and biases of the proposed model primarily revolve around the necessity for specific constraint adjustments when expanding its application beyond low-frequency data generation in NILM to high-frequency data scenarios in NILM or other fields. The model's current design, optimized for NILM's low-frequency power data, may not adequately address the intricacies of high-frequency voltage and current data generation without the implementation of tailored constraints. This requirement for customization may restrict the model's universality and adaptability.

Future research will focus on cross-domain exploration to adapt the proposed model for wider applications. Through targeted experimentation and analysis, we aim to uncover key performance influencers across various environments and develop a tailored constraint formulation methodology. In addition, establishing a broad constraint library will enhance the model's universality, extending its utility across diverse fields and data generation challenges.

APPENDIX

A. Pseudocode of C-TimeGAN

See Algorithm 1.

REFERENCES

[1] R. Ramadan, Q. Huang, O. Bamisile, and A. S. Zalhaf, "Intelligent home energy management using Internet of Things platform based on NILM technique," *Sustain. Energy, Grids Netw.*, vol. 31, Sep. 2022, Art. no. 100785.

[2] G. W. Hart, "Nonintrusive appliance load monitoring," *Proc. IEEE*, vol. 80, no. 12, pp. 1870–1891, Jun. 1992.

[3] J. Liang, S. K. K. Ng, G. Kendall, and J. W. M. Cheng, "Load signature study—Part II: Disaggregation framework, simulation, and applications," *IEEE Trans. Power Del.*, vol. 25, no. 2, pp. 561–569, Apr. 2010.

[4] C. L. Athanasiadis, T. A. Papadopoulos, and D. I. Doukas, "Real-time non-intrusive load monitoring: A light-weight and scalable approach," *Energy Buildings*, vol. 253, Dec. 2021, Art. no. 111523.

[5] T. Hassan, F. Javed, and N. Arshad, "An empirical investigation of V-I trajectory based load signatures for non-intrusive load monitoring," *IEEE Trans. Smart Grid*, vol. 5, no. 2, pp. 870–878, Mar. 2014.

[6] F. Ciancetta, G. Bucci, E. Fiorucci, S. Mari, and A. Fioravanti, "A new convolutional neural network-based system for NILM applications," *IEEE Trans. Instrum. Meas.*, vol. 70, pp. 1–12, 2021.

[7] J. Chen, X. Wang, X. Zhang, and W. Zhang, "Temporal and spectral feature learning with two-stream convolutional neural networks for appliance recognition in NILM," *IEEE Trans. Smart Grid*, vol. 13, no. 1, pp. 762–772, Jan. 2022.

[8] A. Harell, R. Jones, S. Makonin, and I. V. Bajic, "TraceGAN: Synthesizing appliance power signatures using generative adversarial networks," *IEEE Trans. Smart Grid*, vol. 12, no. 5, pp. 4553–4563, Sep. 2021.

[9] H. Guo, J. Lu, P. Yang, and Z. Liu, "Review on key techniques of non-intrusive load monitoring," *Electric Power Autom. Equip., China*, vol. 41, pp. 135–146, Jan. 2021.

[10] D. Murray, L. Stankovic, and V. Stankovic, "An electrical load measurements dataset of united kingdom households from a two-year longitudinal study," *Sci. Data*, vol. 4, no. 1, pp. 1–12, Jan. 2017.

[11] A. Reinhardt et al., "On the accuracy of appliance identification based on distributed load metering data," in *Proc. Sustain. Internet ICT Sustainability (SustainIT)*, Oct. 2012, pp. 1–9.

[12] Z. Liu, L. Zhang, L. Zhang, and T. Ji, "Generation of realistic and high-quality appliance trajectories based on TimeGAN for non-intrusive load monitoring," in *Proc. 6th Int. Conf. Energy, Electr. Power Eng. (CEEPE)*, May 2023, pp. 1500–1504.

[13] D. Egarter, V. P. Bhuvana, and W. Elmenreich, "PALDi: Online load disaggregation via particle filtering," *IEEE Trans. Instrum. Meas.*, vol. 64, no. 2, pp. 467–477, Feb. 2015.

[14] X. Liu, W. He, P. Guo, Y. Yang, T. Guo, and Z. Xu, "Semi-contactless power measurement method for single-phase enclosed two-wire residential entrance lines," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–16, 2022.

[15] M. Nour, J. C. Le Bunetel, P. Ravier, and Y. Raingeaud, "Data augmentation strategies for high-frequency NILM datasets," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–9, 2023.

[16] Z. Du, L. Gao, and X. Li, "A new contrastive GAN with data augmentation for surface defect recognition under limited data," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–13, 2023.

[17] H. Hwang and S. Kang, "Nonintrusive load monitoring using an LSTM with feedback structure," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–11, 2022.

[18] R. Feng, W. Yuan, L. Ge, and S. Ji, "Nonintrusive load disaggregation for residential users based on alternating optimization and downsampling," *IEEE Trans. Instrum. Meas.*, vol. 70, pp. 1–12, 2021.

[19] I. Goodfellow et al., "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 2672–2680.

[20] L. Song, Y. Li, and N. Lu, "ProfileSR-GAN: A GAN based super-resolution method for generating high-resolution load profiles," *IEEE Trans. Smart Grid*, vol. 13, no. 4, pp. 3278–3289, Jul. 2022.

[21] Y. Pan, K. Liu, Z. Shen, X. Cai, and Z. Jia, "Sequence-to-subsequence learning with conditional GAN for power disaggregation," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2020, pp. 3202–3206.

[22] A. M. A. Ahmed, Y. Zhang, and F. Eliassen, "Generative adversarial networks and transfer learning for non-intrusive load monitoring in smart grids," in *Proc. IEEE Int. Conf. Commun., Control, Comput. Technol. Smart Grids (SmartGridComm)*, Nov. 2020, pp. 1–7.

[23] L. Huang, W. Pan, Y. Zhang, L. Qian, N. Gao, and Y. Wu, "Data augmentation for deep learning-based radio modulation classification," *IEEE Access*, vol. 8, pp. 1498–1506, 2019.

[24] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.

[25] X. He, X. Jiang, P. Zhang, X. Gao, and Z. Li, "SVM based parameter estimation of relay protection reliability with small samples," *Power Syst. Technol., China*, vol. 39, pp. 1432–1437, May 2015.

[26] S. Wang, H. Chen, X. Li, and X. Shu, "Conditional variational automatic encoder method for stochastic scenario generation of wind power and photovoltaic system," *Power Syst. Technol., China*, vol. 42, pp. 1860–1869, Jun. 2018.

[27] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," 2015, *arXiv:1511.06434*.

[28] J. Yoon, D. Jarrett, and M. Van der Schaar, "Time-series generative adversarial networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 5509–5519.

[29] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, Jul. 2001.

[30] H. J. Shin, D.-H. Eom, and S.-S. Kim, "One-class support vector machines—An application in machine fault detection and classification," *Comput. Ind. Eng.*, vol. 48, no. 2, pp. 395–408, Mar. 2005.

[31] J. Z. Kolter and M. J. Johnson, "REDD: A public data set for energy disaggregation research," *Sustkdd*, vol. 25, pp. 59–62, 2011.

[32] C. Zhang, M. Zhong, Z. Wang, N. Goddard, and C. Sutton, "Sequence-to-point learning with neural networks for non-intrusive load monitoring," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 2604–2611.

[33] B. Zhao, K. He, L. Stankovic, and V. Stankovic, "Improving event-based non-intrusive load monitoring using graph signal processing," *IEEE Access*, vol. 6, pp. 53944–53959, 2018.

[34] A. Borji, "Pros and cons of GAN evaluation measures," *Comput. Vis. Image Understand.*, vol. 179, pp. 41–65, Feb. 2019.

[35] M. Heusel et al., "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6626–6637.

[36] R. Julien, P. Gabriel, D. Julie, and B. Marc, "Wasserstein barycenter and its application to texture mixing," in *Proc. Scale Space Variational Methods Comput. Vis. (SSVM)*, 2011, pp. 435–446.

[37] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 86, pp. 2579–2605, 2008.

[38] F. B. Bryant and P. R. Yarnold, "Principal-components analysis and exploratory and confirmatory factor," in *Proc. Reading Understand. Multivariate Statist.*, 1995, pp. 99–136.

[39] J. Kelly and W. Knottenbelt, "Neural NILM: Deep neural networks applied to energy disaggregation," in *Proc. 2nd ACM Int. Conf. Embedded Syst. Energy-Efficient Built Environments*, 2015, pp. 55–64.

Z. G. Liu received the B.S. degree in electrical engineering from Guangdong University of Technology, Guangzhou, China, in 2021. He is currently pursuing the Ph.D. degree with the School of Electric Power Engineering, South China University of Technology, Guangzhou.

His research interests include applications of deep learning and signal processing in nonintrusive load monitoring.

T. Y. Ji (Senior Member, IEEE) received the B.Eng. degree in information engineering, the B.A. degree in English, and the M.Sc. degree in signal and information processing from Xi'an Jiaotong University, Xi'an, China, in 2003, 2003, and 2006, respectively, and the Ph.D. degree in electrical engineering and electronics from the University of Liverpool, Liverpool, U.K., in 2009.

From 2010 to 2011, she worked as a Research Associate with the University of Liverpool. Since 2012, she has been working with the School of Electric Power Engineering, South China University of Technology, Guangzhou, China, as an Associate Professor and then a Professor. Her research interests include signal and information processing and its application in power systems.

J. W. Chen received the B.S. degree in electrical engineering and its automation from Fuzhou University, Fuzhou, China, in 2021. He is currently pursuing the M.S. degree in electrical engineering with South China University of Technology, Guangzhou, China.

His research interests focus on nonintrusive load monitoring and electric load forecasting.

L.J. Zhang received the B.S. degree in electrical engineering from South China University of Technology, Guangzhou, China, in 2022, where he is currently pursuing the M.S. degree.

His research interests focus on applications of machine learning and data science in nonintrusive load monitoring.

L.L. Zhang (Member, IEEE) received the B.E. degree in electrical engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2009, and the Ph.D. degree in electrical engineering from South China University of Technology, Guangzhou, China, in 2014.

Then, he worked as a Research Fellow and subsequently a Lecturer with South China University of Technology. He is currently an Associate Professor with the School of Electric Power Engineering, South China University of Technology. His research interests include fault diagnosis, relay protection, and signal processing in power systems.

Q. H. Wu (Life Fellow, IEEE) received the M.Sc. (Eng.) degree in electrical engineering from the Huazhong University of Science and Technology, Wuhan, China, in 1981, and the Ph.D. degree in electrical engineering from The Queen's University of Belfast (QUB), Belfast, U.K., in 1987.

From 1981 to 1984, he was appointed as a Lecturer in electrical engineering in the university. He worked as a Research Fellow and subsequently a Senior Research Fellow with QUB from 1987 to 1991. In 1991, he joined the Department of Mathematical Sciences, Loughborough University, Loughborough, U.K., as a Lecturer, and subsequently, he was appointed as a Senior Lecturer. In September 1995, he joined The University of Liverpool, Liverpool, U.K., to take up his appointment to the Chair of Electrical Engineering, Department of Electrical Engineering and Electronics. He is currently with the School of Electric Power Engineering, South China University of Technology, Guangzhou, China, as a Distinguished Professor and the Director of the Energy Research Institute. He has authored or coauthored more than 700 technical publications, including 370 journal articles, 20 book chapters, and four research monographs published by Springer. His research interests include nonlinear adaptive control, mathematical morphology, evolutionary computation, power quality, and power system control and operation.